Python Nmap Port Scanner

Contents

Python Nmap Port Scanner 1
What is Nmap? 1
Tutorial 1: Install Nmap 2
python-nmap 2
Tutorial 2: Nmap with Python 2
Tutorial 3: Nmap in Python with GUI 5
Assignment Submission 10

Time required: 60 minutes

NOTE: Complete this tutorial in Kali Linux.

What is Nmap?

Nmap (Network Mapper) is a security scanner, originally written by Gordon Lyon (also known by his pseudonym Fyodor Vaskovich), and used to discover hosts and services on a computer network, thereby building a map of the network. To accomplish its goal, Nmap sends specially crafted packets to the target host(s) and then analyzes their responses.

Some of the useful Nmap features include:

- **Host Discovery**: This enables to identify hosts on any network. For example, listing the hosts that respond to TCP and/or ICMP requests or have a particular **port** open.
- **Port Scanning**: Enumerating (counting and listing one by one) all the open ports on the target hosts.
- **Version Detection**: Interrogating network services on remote devices to determine application name and version number.
- **OS Detection**: Determining the operating system and hardware characteristics of the network devices.

- **Scriptable Interaction with the target**: Using Nmap Scripting Engine (NSE) and Lua programming language, we can easily write sripts to perform operations on the network devices.

Tutorial 1: Install Nmap

The nmap install now includes the GUI interface for Nmap known as **zenmap**. Nmap is already install in Kali Linux.

For Windows and Mac OS X, download and install Nmap→ https://nmap.org/download.html

python-nmap

The **python-nmap** library is a wrapper around the **nmap** program. You must have **nmap** installed first. This library allows you to create any type of CLI or GUI program using **nmap**.

Before we start using Nmap, let's install the python-nmap module:

```
# Windows: pip install python-nmap
# Linux: sudo pip3 install python-nmap
```

Tutorial 2: Nmap with Python

Create a Python file named **nmap_ping_scan_cli.py**

```python
#!/usr/bin/env python3
"""
    Name: nmap_ping_scan_gui.py
    Author:
    Created:
    Purpose: Use Python nmap wrapper to scan network
"""
# Windows: pip install python-nmap
# Linux: sudo pip3 install python-nmap
import nmap


def main():
    print(" +-------------------------------------------------+")
    print(" |------     Python nmap Network Scanner     ------|")
    print(" +-------------------------------------------------+")

    # Change to default value of your network
    local_network = "192.168.9.0/24"
    # Get network address, if blank use local_network
    network = input(
        " Enter network address (192.168.1.0/24): ") or local_network
    scan(network)
```

```python
# ------------------------ SCAN ------------------------------------------#
def scan(network):
    """Create Nmap port scanner object"""
    nm = nmap.PortScanner()

    print(f" Nmap Version: {nm.nmap_version()}")
    print(" Scanning the network . . .")

    # Set target and arguments
    # You can get common settings from Zenmap
    # -sn - Ping scan
    nm.scan(
        hosts=network,
        arguments="-sn"
    )

    # Counter to track which host we are scanning
    num_hosts = 0
    for host in nm.all_hosts():
        # Loop through all hosts one at a time
        # Get scan information from nmap dictionary
        host = nm[host].get("addresses").get("ipv4")
        state = nm[host].get("status").get("state")
        # Display the host IP and status
        print(f" {host} \t{state}")
        num_hosts += 1

    print(f" Number of hosts: {num_hosts}")
    # Get and display scan statistics
    scan_stats = nm.scanstats()
    print(f" Elapsed: {scan_stats.get('elapsed')} seconds\n")


if __name__ == '__main__':
    main()
```

Use bridged adapter for network scan.

In Linux → Terminal → **sudo python3 nmap_ping_scan_cli.py**

Example run using bridged adapter:

```
+-----------------------------------------------+
|------     Python nmap Network Scanner     ------|
+-----------------------------------------------+
Enter network address (192.168.1.0/24):
Nmap Version: (7, 93)
Scanning the network . . .
192.168.9.1:   up
192.168.9.10:  up
192.168.9.103: up
192.168.9.104: up
192.168.9.110: up
192.168.9.112: up
192.168.9.130: up
192.168.9.134: up
192.168.9.137: up
192.168.9.139: up
192.168.9.140: up
192.168.9.245: up
Number of hosts: 12
Elapsed: 2.64 seconds
```

Tutorial 3: Nmap in Python with GUI

Same program with a Tkinter GUI.

In Linux → Terminal → **sudo python3 nmap_ping_scan_gui.py**

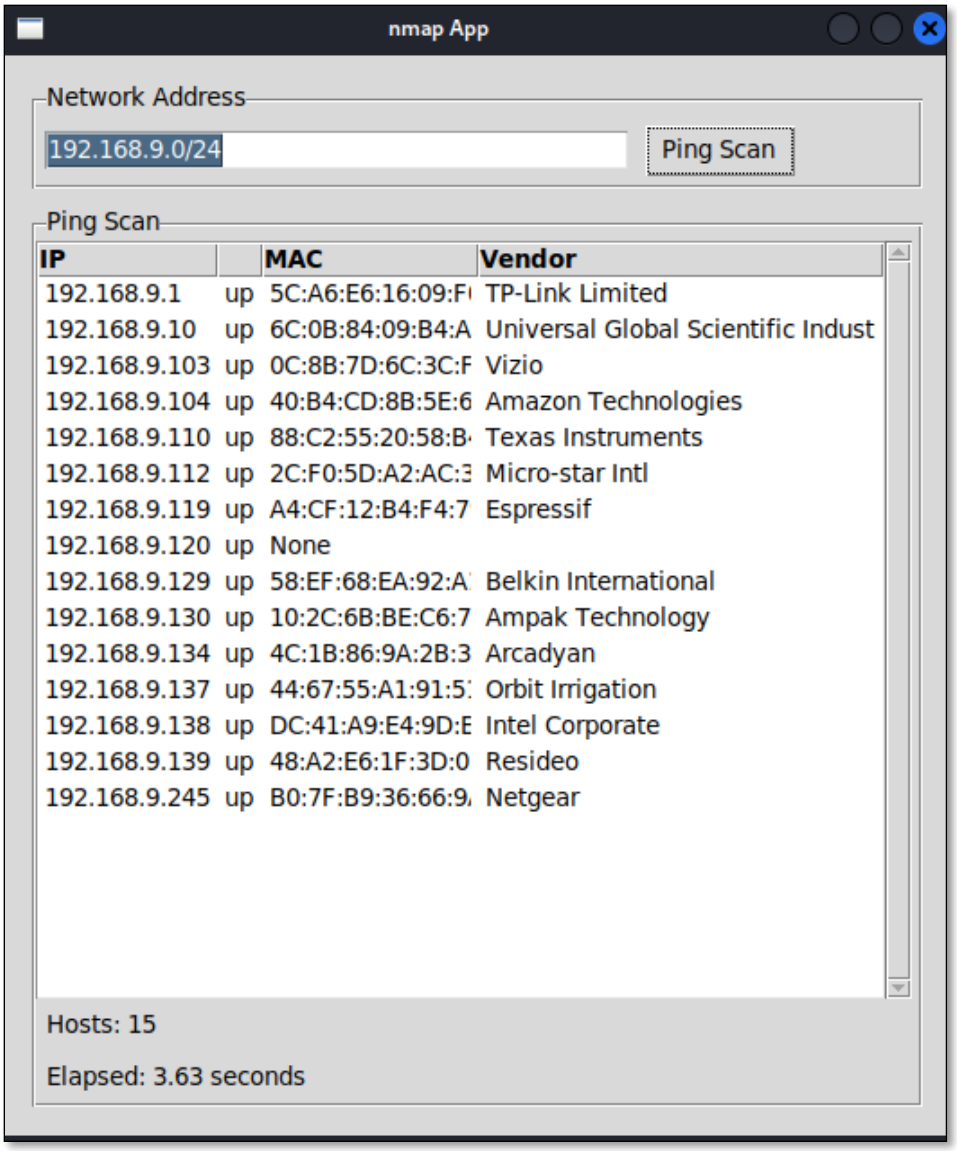
```python
#!/usr/bin/env python3
"""
    Name: nmap_ping_scan_gui.py
    Author:
    Created:
    Purpose: Purpose: Use Python nmap wrapper to scan network
"""

from tkinter import *
from tkinter.ttk import *
# Windows: pip install python-nmap
# Linux: sudo pip3 install python-nmap
import nmap


class NmapScanner:

    def __init__(self):
        """ Initialize program """
        self.window = Tk()
        self.window.title("nmap App")
        self.window.geometry("525x600")
        self.window.config(padx=10, pady=10)

        self.create_widgets()
        self.create_treeview()
        mainloop()
```

```python
# ------------------------- SCAN -------------------------------------------#
    def scan(self, *args):
        # Return a list of tuples from treeview
        items = self.tree.get_children()

        # Iterate through list to delete all items in the treeview
        for item in items:
            self.tree.delete(item)

        # Initialize create Nmap port scanner object
        nm = nmap.PortScanner()
        self.network = self.entry_network_address.get()
        # Set target and arguments
        # You can get common settings from Zenmap
        # -sn - Ping scan
        nm.scan(
            hosts=self.network,
            arguments="-sn"
        )

        num_hosts = 0
        for hosts in nm.all_hosts():
            # Get scan information from nmap dictionary
            host = nm[hosts].get("addresses").get("ipv4")
            state = nm[hosts].get("status").get("state")
            mac_address = nm[hosts].get("addresses").get("mac")
            # If the device does not have a MAC address
            # it will not have a MAC vendor.
            try:
                mac_vendor = list(nm[hosts].get('vendor').values())[0]
            except:
                # If there isn't a MAC address or the MAC address lookup
                # fails, leave it empty
                mac_vendor = ""

            self.tree.insert("", "end", text=num_hosts, values=(
                host, state, mac_address, mac_vendor)
            )
            num_hosts += 1

        # Get scan statistics
        scan_stats = nm.scanstats()
        self.lbl_hosts_value.config(text=f"Hosts: {num_hosts}")
        self.lbl_elapsed_value.config(
            text=f"Elapsed: {scan_stats.get('elapsed')} seconds")
```

```python
# ------------------------- CREATE WIDGETS --------------------------------#
    def create_widgets(self):
        """Create and place GUI widgets"""
        # Create widgets
        # Create frames
        self.entry_frame = LabelFrame(self.window, text="Network Address")
        self.display_frame = LabelFrame(self.window, text="Ping Scan")

        # Fill the frame to the width of the window
        self.entry_frame.pack(fill=X)
        self.display_frame.pack(fill=X)
        # Keep the frame size regardless of the widget sizes
        self.entry_frame.pack_propagate(False)
        self.display_frame.pack_propagate(False)

        self.entry_network_address = Entry(self.entry_frame, width=40)
        # Set this to your default network address
        self.entry_network_address.insert(
            END, string="192.168.9.0/24")
        # Select all text in entry
        self.entry_network_address.selection_range(0, END)
        self.entry_network_address.focus_set()

        self.btn_scan = Button(
            self.entry_frame,
            text="Ping Scan",
            command=self.scan
        )

        self.lbl_hosts_value = Label(self.display_frame, text="Hosts:")
        self.lbl_elapsed_value = Label(self.display_frame, text="Elapsed:")
        self.lbl_hosts_value.grid(row=1, column=0, sticky=W)
        self.lbl_elapsed_value.grid(row=2, column=0, sticky=W)

        # Enter key will activate the scan method
        self.window.bind('<Return>', self.scan)
        self.window.bind('<KP_Enter>', self.scan)

        # Place Widgets
        self.entry_network_address.grid(
            row=1, column=1, columnspan=2, sticky=W)
        self.btn_scan.grid(row=1, column=3)

        self.entry_frame.pack_configure(padx=5, pady=5)
        self.display_frame.pack_configure(padx=5, pady=5)
        # Set padding for all widgets
        for child in self.entry_frame.winfo_children():
            child.grid_configure(padx=5, pady=5)
        for child in self.display_frame.winfo_children():
            child.grid_configure(padx=5, pady=5)
```

```python
# -------------------------- TREEVIEW AND SCROLLBAR ------------------------#
    def create_treeview(self):
        """Setup tree view for display"""
        # Create treeview
        self.tree = Treeview(
            self.display_frame,
            height=20,
            columns=("ip", "state", "mac", "vendor"),
            style="Treeview",
            show="headings",
            selectmode="browse"
        )
        # Setup the columns
        self.tree.column("ip", width=100)
        self.tree.column("state", width=25)
        self.tree.column("mac", width=120)
        self.tree.column("vendor", width=225)

        # Setup the heading text visible at the top of the column
        self.tree.heading("ip", text="IP", anchor=W)
        self.tree.heading("state", text="", anchor=W)
        self.tree.heading("mac", text="MAC", anchor=W)
        self.tree.heading("vendor", text="Vendor", anchor=W)

        # Grid the tree
        self.tree.grid(row=0, column=0)

        # Create scrollbar for treeview
        self.scrollbar = Scrollbar(
            self.display_frame,
            orient="vertical",
            command=self.tree.yview
        )

        # Set scroll bar to scroll vertically and attach to the tree
        self.tree.configure(yscroll=self.scrollbar.set)

        # Grid scrollbar just to the right of the tree
        # sn (SouthNorth) expands scrollbar to height of tree
        self.scrollbar.grid(row=0, column=1, sticky="sn")


# Create program object
nmap_scanner = NmapScanner()
```

Example run using bridged adapter in Windows:



Assignment Submission

1. Attach the code.
2. Attach a screenshot showing a successful run of the program.
3. Submit the assignment in Blackboard.